

Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Teoría

Ignacio Sammartino

Índice

0 - Introducción a los Microcontroladores	3
0.1 - Integración y SoC	3
0.2 - Clasificación de MCUs	3
0.3 - Conexionado y Funcionamiento Básico ...	3
0.4 - Consumo y Energía	3
0.5 - Ecosistema de Desarrollo	3
0.6 - Introducción a Kinetis K64F	3
0.6.1 - La Familia Kinetis	3
0.6.2 - Microcontrolador K64	3
0.6.3 - Placa de Evaluación: FRDM-K64F ..	4
0.6.4 - Toolchain: MCUXpresso	4
1 - Clase 1	5
1.1 - DataBus/DataAddress	5
1.2 - GPIO	6
1.2.1 - Readback	7
1.2.2 - ActiveLow/ActiveHigh	7
1.2.3 - Propiedades de un pin	8
1.2.4 - Configuraciones de pines con MOSFETS	9
1.3 - GPIO del K64	9
1.3.1 - PDDR	10
1.3.2 - PSOR	11
1.3.3 - PCOR	11
1.3.4 - PTOR	11
1.3.5 - PDIR	12
1.4 - PCR	12

0 - Introducción a los Microcontroladores

0.1 - Integración y SoC

Históricamente, los sistemas microprocesados requerían múltiples integrados (CPU, RAM, ROM, Clock, etc.) . Los **Microcontroladores (MCU)** actuales integran todos estos bloques en un solo chip, conocido como **System on Chip (SoC)** :

- **CPU:** Microprocesador central.
- **Memoria:** RAM para datos y Flash/ROM para programa.
- **Periféricos:** Módulos de hardware (Timers, ADC, UART) configurables mediante registros que funcionan en paralelo a la CPU, permitiendo multitarea real .

0.2 - Clasificación de MCUs

Se eligen y distinguen principalmente por :

- **CPU:** Arquitecturas de 8, 16 o 32 bits.
- **Memoria:** Cantidad de RAM y FLASH disponible.
- **Encapsulado:** Número y tipo de pines físicos.
- **Aplicación:** Bajo costo, alta performance o bajo consumo (**Low Power**).

0.3 - Conexionado y Funcionamiento Básico

- **Alimentación:** Pines VDD y GND. La tendencia actual es bajar la tensión (3.3V, 1.8V) para reducir el consumo .
- **Clock:** Provee la señal de sincronismo. Muchos poseen PLL internos para multiplicar la frecuencia del oscilador .
- **Reset:** Asegura que el programa inicie desde una condición predefinida (pin nRESET) .
- **Entradas/Salidas (I/O):**
- **GPIO:** Pines digitales configurables (entrada/salida, pull-up/down, etc.) .
- **Analógicos:** Conversores A/D y D/A para el mundo real .

0.4 - Consumo y Energía

El consumo se divide en potencia estática y dinámica, dominando generalmente la **dinámica** . Es fundamental el uso de modos **Sleep** para ahorrar energía en sistemas a batería, teniendo en cuenta que a menor consumo, mayor es el tiempo de despertar (**wakeup**).

0.5 - Ecosistema de Desarrollo

Para trabajar con un MCU se requiere [19-22]:

1. **Documentación:** Datasheet y Reference Manual.
2. **Evaluation Board:** Placa de desarrollo con el MCU.
3. **Toolchain:** IDE, compilador y programador/debugger.
4. **SDK:** Librerías y ejemplos de código.

0.6 - Introducción a Kinetis K64F

0.6.1 - La Familia Kinetis

Kinetis es una familia de microcontroladores de **32 bits de NXP** basada en el núcleo **ARM Cortex-M** .

0.6.2 - Microcontrolador K64

El modelo específico de estudio es el **MK64FN1M0VLL12** . Sus características principales son :

- **Core:** ARM Cortex-M4 a 120 MHz con FPU (Unidad de Punto Flotante).
- **Memoria:** 1 MB de Flash y 256 KB de SRAM.
- **Encapsulado:** 100 LQFP.

0.6.3 - Placa de Evaluación: FRDM-K64F

Es una plataforma de bajo costo que incluye :

- **Conectividad:** Ethernet, USB (Host/Device) y slot para MicroSD (SDHC).
- **Sensores:** Acelerómetro y magnetómetro integrados.
- **Interfaz Humana:** LED RGB y 2 pulsadores (SW2 y SW3).
- **Depuración:** Programador incorporado **OpenSDAv2**.

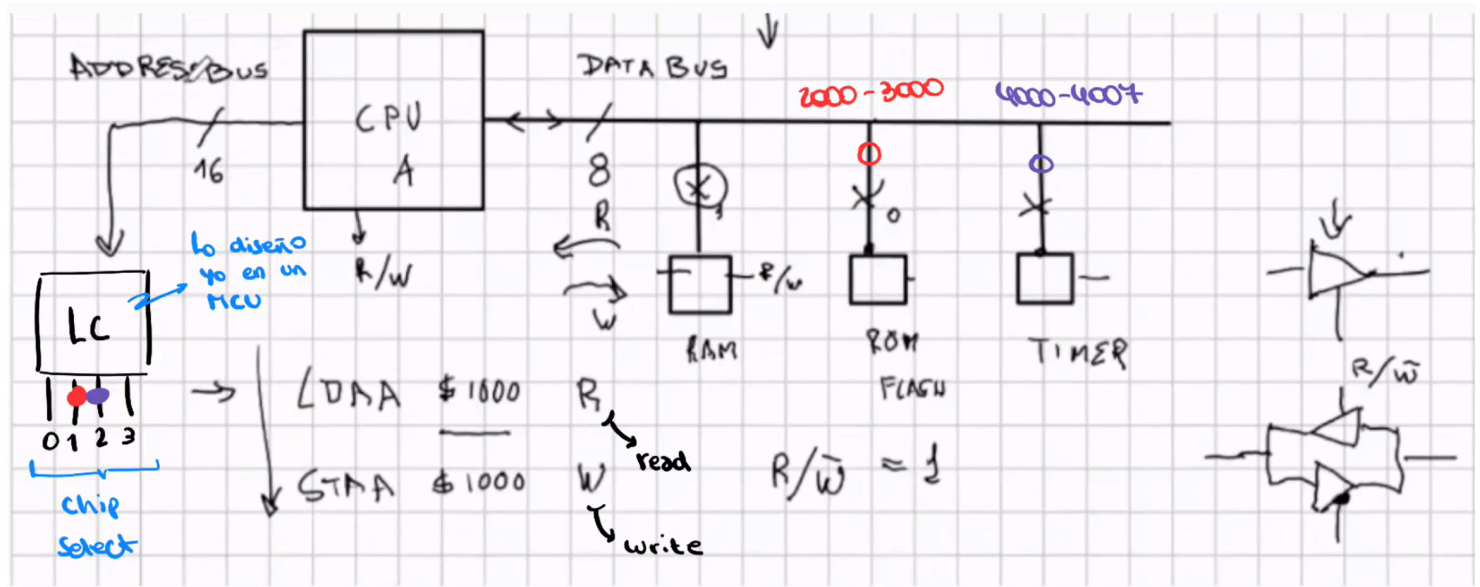
0.6.4 - Toolchain: MCUXpresso

El IDE oficial de NXP para esta plataforma es **MCUXpresso**, basado en Eclipse . Para su funcionamiento con la familia K64, es indispensable instalar el **SDK específico** de la placa FRDM-K64F .

Nota sobre Registros: El control de los periféricos en el K64 se realiza mediante la escritura en registros de 32 bits, como el **Pin Control Register** (PCR), que define la multiplexación (MUX) y características eléctricas de cada pin .

1 - Clase 1

1.1 - DataBus/DataAddress



Con LDAA \$1000 cargo el contenido de la dirección de memoria al adder A en la CPU a través del DataBus.

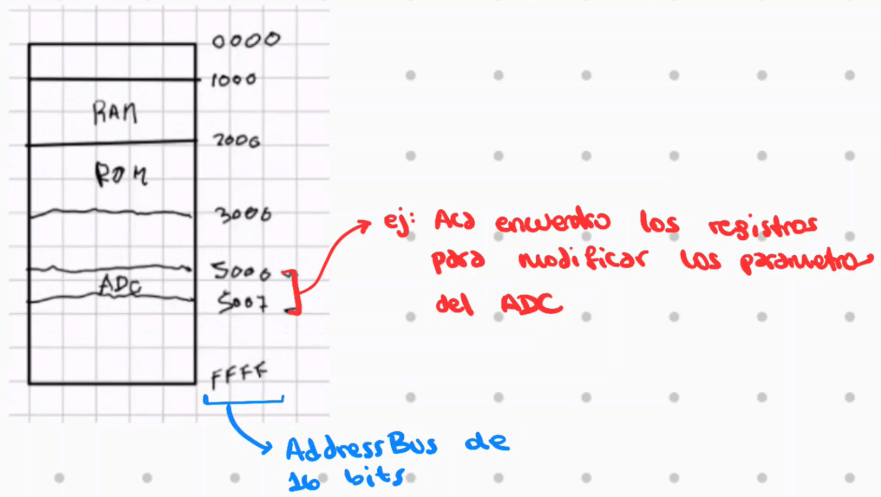
Para impedir que los periféricos se superpongan, se utilizan buffers a través **READ** and **WRITE** (R/W). Estos R/W se encuentran para cada periférico determinando si se encuentran en funcionamiento o no.

Baja impedancia = 1

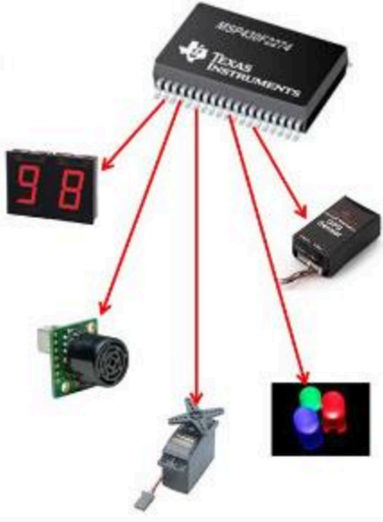
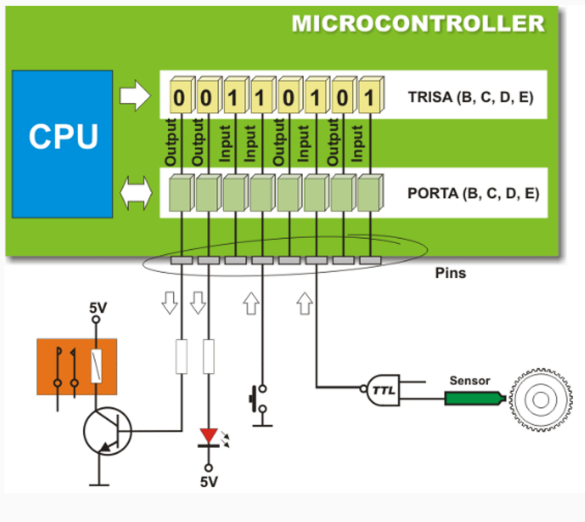
Alta impedancia = 0

El AddressBus tiene información de con quien voy a hablar, en este caso con la posición \$1000 de memoria. El AddressBus se conecta a **Logica de Decodificación Debugable (LC)** que es una lógica combinacional que son una serie de salidas. Cuando el AddressBus se encuentra en cierto **rango** se activan **ciertas** salidas. Las salidas se llaman ChipSelect o también conocido como FLASH. Por ejemplo, en caso de que la información del AddressBus se encuentre entre \$1000-\$2000 viniendo de la RAM, se activa el CS_{RAM} (Chip Select de la RAM).

La **Logica de Decodificación Debugable (LC)** la diseñó uno cuando diseñó un MCU.



1.2 - GPIO



Para que el AdressBus pueda diferenciar los diferentes registros del GPIO, estos tienen diferentes posiciones de memoria. Cada Registro tendrá su propio ChipSelect.

En este ejemplo, P1DIR determina si el pin se encuentra leyendo o escribiendo un dato. Pensar que en la imagen se replica por cada bit.

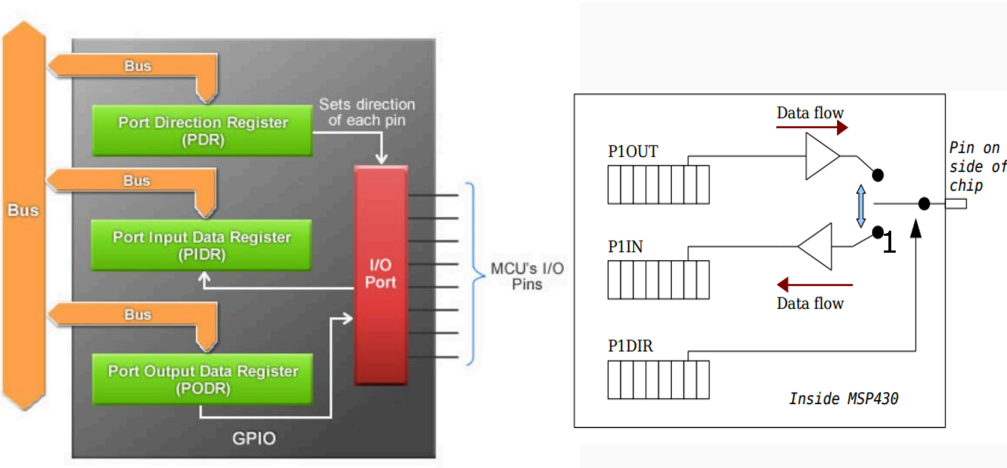


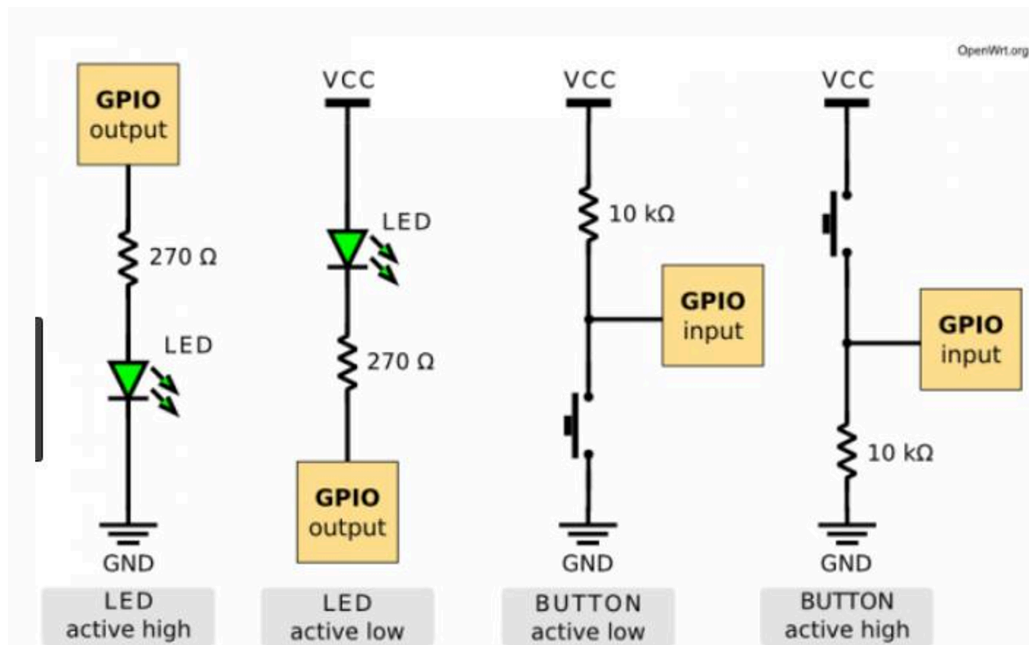
Figura 5: Ejemplo Generico

1.2.1 - Readback

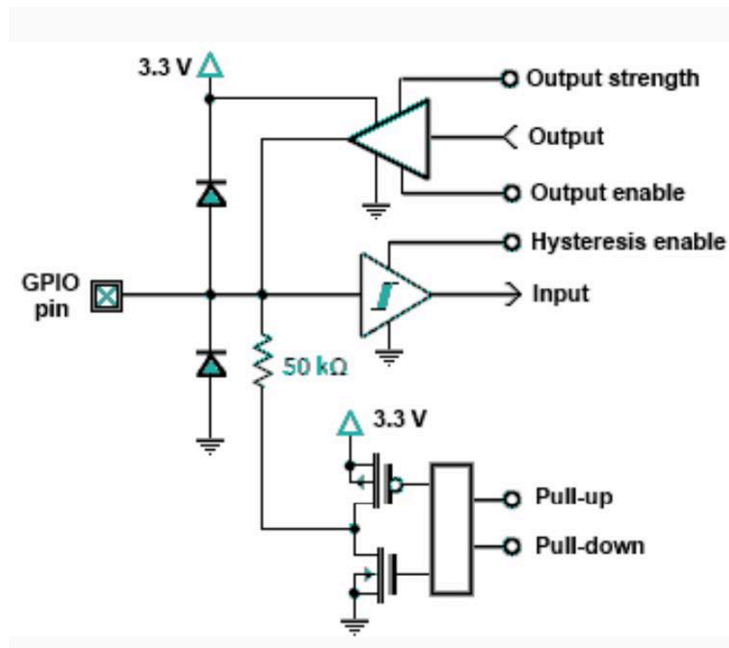
Es una técnica para poder leer el estado de un pin utilizando otro. Para esto, uno de ellos se lo declara OUTPUT y el otro INPUT.



1.2.2 - ActiveLow/ActiveHigh



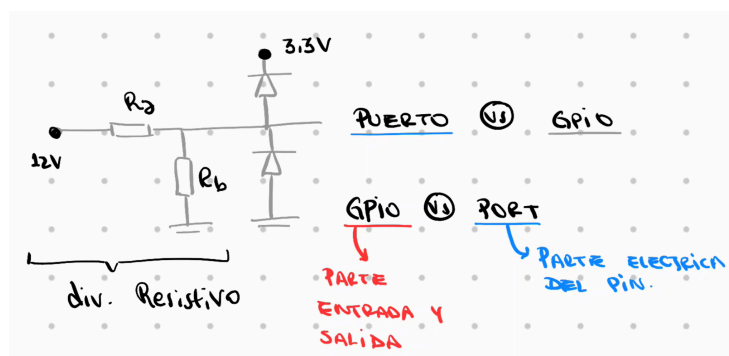
1.2.3 - Propiedades de un pin



Cuentan con diodos en caso de error con las tensiones comenzando a conducir (Por encima de V_{DD} y debajo de V_{EE}). Además, puedo configurar una resistencia de PULL-DOWN o PULL-UP.

Puedo pifiar con software configurando INPUTS y OUTPUTS. Una forma de detectar si el pin es entrada o salida, es colocar la punta del osciloscopio y acercar un dedo. Si esta configurado como OUTPUT, no se debería ver nada al presentar una **baja** impedancia mientras que de estar configurada como INPUT se debería observar una senoidal inducida de 50 Hz al presentar un **alta** impedancia.

En el caso de tener entradas al pin mayores o por debajo del rango del pin, es necesario aplicar un divisor resistivo y un par de diodos en caso de que falle la conexión del divisor resistivo.



Drive Strength: Es la máxima corriente que puede entregar el pin a la carga. Por lo tanto el DS fija la máxima carga que se puede poner para una velocidad dada sin afectar la integridad de la señal.

1.2.4 - Configuraciones de pines con MOSFETS

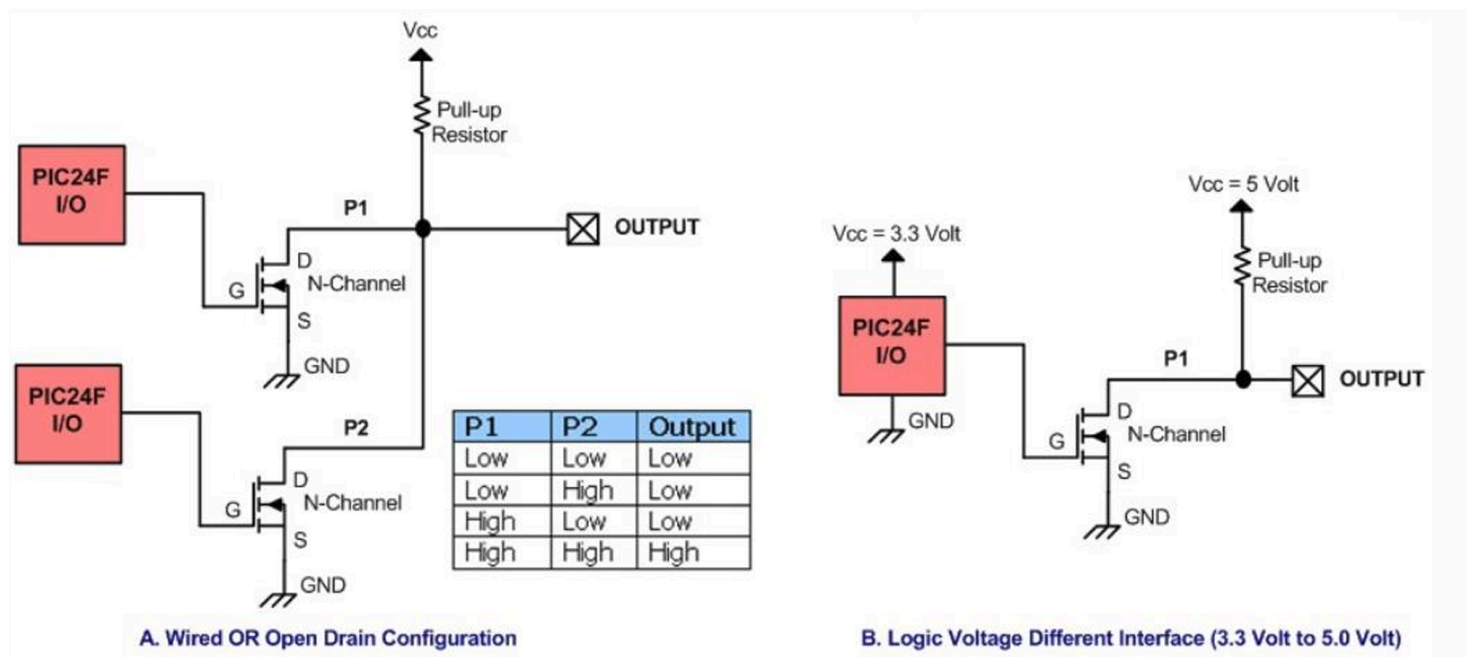


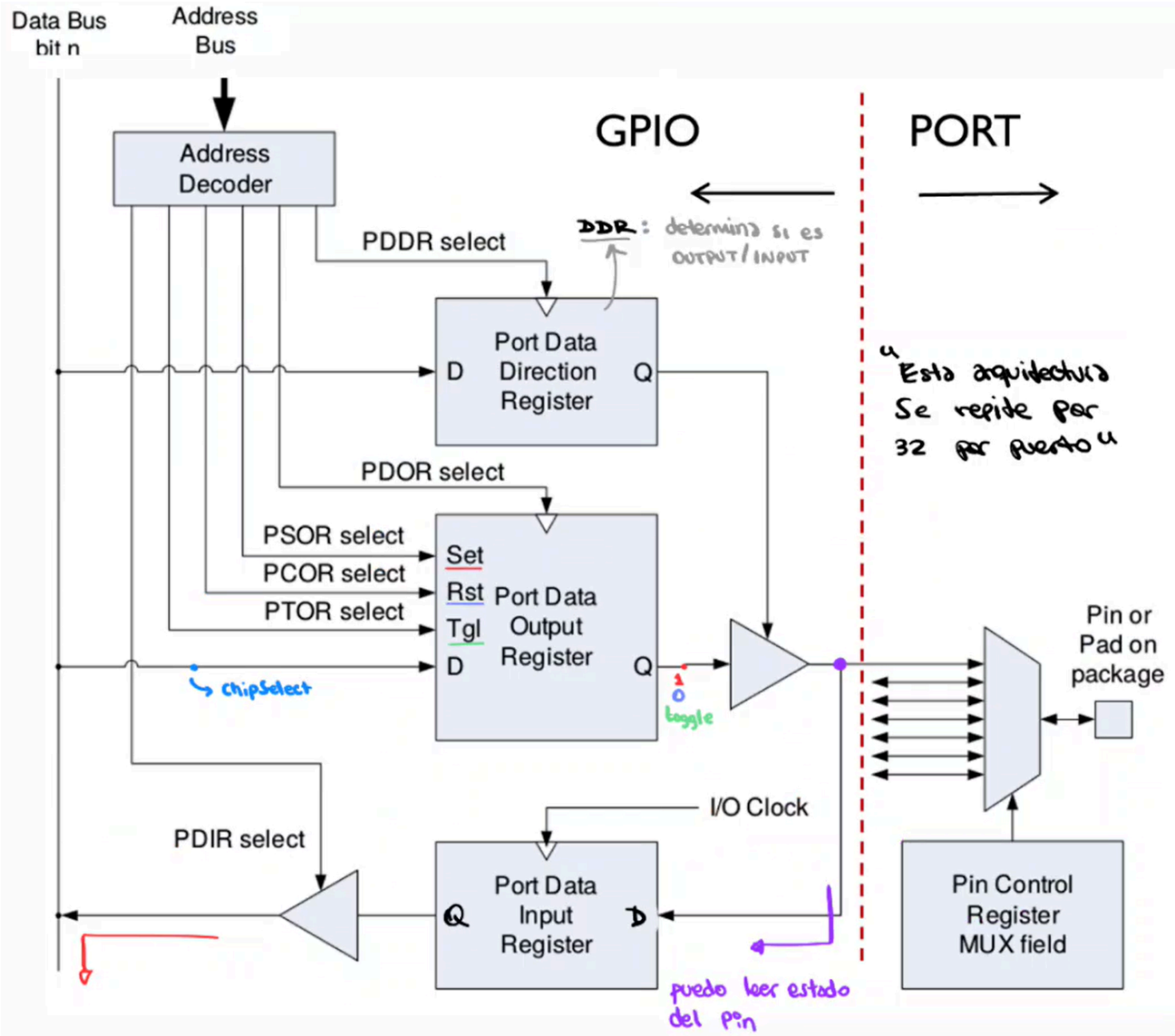
Figura 10: Como implementar logica digital con MOSFETS y como elevar la tensiones de salida

1.3 - GPIO del K64

GPIO contiene 5 Puertos de 32 bits c/u

1. PORTA [31:0]
2. PORTB [31:0]
3. PORTC [31:0]
4. PORTD [31:0]
5. PORTE [31:0]

- Port Data Output Register (GPIOX_PDOR) [31:0]
- Port Set Output Register (GPIOX_PSOR) [31:0]
- Port Clear Output Register (GPIOX_PCOR) [31:0]
- Port Toggle Output Register (GPIOX_PTOR) [31:0]
- Port Data Input Register (GPIOX_PDIR) [31:0]
- Port Data Direction Register (GPIOX_PDDR) [31:0]



1.3.1 - PDDR

Address: Base address + 14h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_PDDR field descriptions

Field	Description
PDD	Port Data Direction Configures individual port pins for input or output. 0 Pin is configured as general-purpose input, for the GPIO function. 1 Pin is configured as general-purpose output, for the GPIO function.

1.3.2 - PSOR

Address: Base address + 4h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																															
W	PTSO																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_PSOR field descriptions

Field	Description
PTSO	Port Set Output Writing to this register will update the contents of the corresponding bit in the PDOR as follows: 0 Corresponding bit in PDORn does not change. 1 Corresponding bit in PDORn is set to logic 1.

1.3.3 - PCOR

Address: Base address + 8h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																															
W	PTCO																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_PCOR field descriptions

Field	Description
PTCO	Port Clear Output Writing to this register will update the contents of the corresponding bit in the Port Data Output Register (PDOR) as follows: 0 Corresponding bit in PDORn does not change. 1 Corresponding bit in PDORn is cleared to logic 0.

1.3.4 - PTOR

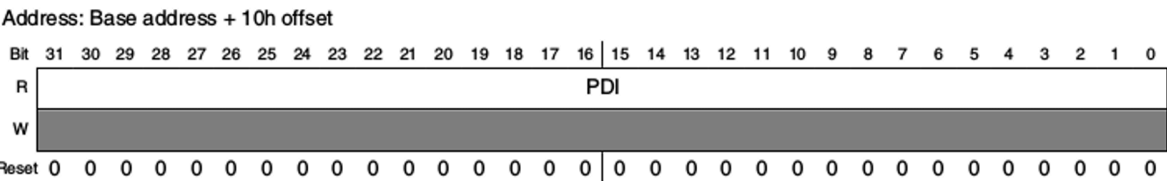
Address: Base address + Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																															
W	PTTO																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_PTOR field descriptions

Field	Description
PTTO	Port Toggle Output Writing to this register will update the contents of the corresponding bit in the PDOR as follows: 0 Corresponding bit in PDORn does not change. 1 Corresponding bit in PDORn is set to the inverse of its existing logic state.

1.3.5 - PDIR



GPIOx_PDIR field descriptions

Field	Description
PDI	Port Data Input Reads 0 at the unimplemented pins for a particular device. Pins that are not configured for a digital function read 0. If the Port Control and Interrupt module is disabled, then the corresponding bit in PDIR does not update. 0 Pin logic level is logic 0, or is not configured for use by digital function. 1 Pin logic level is logic 1.

1.4 - PCR

Cada pin de cada puerto es configurable mediante un registro de 32 bits llamado **Pin Control Register (PCR)**

PORTX_PCRn
donde X=A,B,C,D,E n=0:31

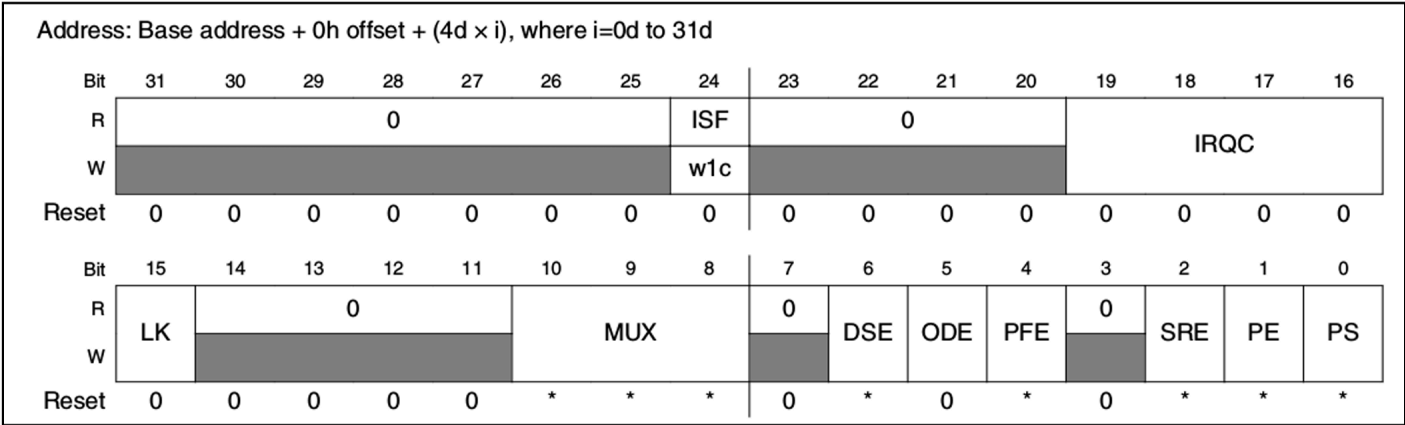


Figura 17: Algo confuso, existen w1c que indican que se debe escribir un 1 para colocar un 0 en el bit deseado.

PORT memory map

Absolute address (hex)	Register name	Width (in bits)
4004_9000	Pin Control Register n (PORTA_PCR0)	32
4004_9004	Pin Control Register n (PORTA_PCR1)	32
4004_9008	Pin Control Register n (PORTA_PCR2)	32
4004_900C	Pin Control Register n (PORTA_PCR3)	32
4004_9010	Pin Control Register n (PORTA_PCR4)	32
4004_9014	Pin Control Register n (PORTA_PCR5)	32
4004_9018	Pin Control Register n (PORTA_PCR6)	32
4004_901C	Pin Control Register n (PORTA_PCR7)	32
4004_9020	Pin Control Register n (PORTA_PCR8)	32
4004_9024	Pin Control Register n (PORTA_PCR9)	32
4004_9028	Pin Control Register n (PORTA_PCR10)	32
4004_902C	Pin Control Register n (PORTA_PCR11)	32
4004_9030	Pin Control Register n (PORTA_PCR12)	32
4004_9034	Pin Control Register n (PORTA_PCR13)	32
4004_9038	Pin Control Register n (PORTA_PCR14)	32
4004_903C	Pin Control Register n (PORTA_PCR15)	32
4004_9040	Pin Control Register n (PORTA_PCR16)	32
4004_9044	Pin Control Register n (PORTA_PCR17)	32
4004_9048	Pin Control Register n (PORTA_PCR18)	32
4004_904C	Pin Control Register n (PORTA_PCR19)	32
4004_9050	Pin Control Register n (PORTA_PCR20)	32
4004_9054	Pin Control Register n (PORTA_PCR21)	32
4004_9058	Pin Control Register n (PORTA_PCR22)	32
4004_905C	Pin Control Register n (PORTA_PCR23)	32

4004_9060	Pin Control Register n (PORTA_PCR24)	32
4004_9064	Pin Control Register n (PORTA_PCR25)	32
4004_9068	Pin Control Register n (PORTA_PCR26)	32
4004_906C	Pin Control Register n (PORTA_PCR27)	32
4004_9070	Pin Control Register n (PORTA_PCR28)	32
4004_9074	Pin Control Register n (PORTA_PCR29)	32
4004_9078	Pin Control Register n (PORTA_PCR30)	32
4004_907C	Pin Control Register n (PORTA_PCR31)	32
4004_9080	Global Pin Control Low Register (PORTA_GPCLR)	32
4004_9084	Global Pin Control High Register (PORTA_GPCHR)	32
4004_90A0	Interrupt Status Flag Register (PORTA_ISFR)	32
4004_90C0	Digital Filter Enable Register (PORTA_DFER)	32
4004_90C4	Digital Filter Clock Register (PORTA_DFCR)	32
4004_90C8	Digital Filter Width Register (PORTA_DFWR)	32

Esto se repite para los puertos B,C,D,E

Figura 18: Ejemplo de PCR para el puerto A

```
gpio.c
1 int variableValida;
2 int _contador;
3 int valor2;
```